

Context Lake

Cross-Session Context Persistence Engine

Design & Implementation Document

Project: petemart-agentic-framework

Platform: Windows | Runtime: Python 3.10+ | Dependencies: Zero (stdlib)

Date: June 2026

Architecture Summary

What is Context Lake?

A file-based, zero-dependency engine that persists AI coding session context across window closes, compaction, and reboots. Every new opencode window automatically loads the latest project state -- pipeline progress, git info, and a full conversation summary.

Capture Triggers

- session close / compaction (auto)
- new window open (auto-load)
- manual capture (on-demand)

What Is Captured

- Pipeline: 69% | 8/16 agents done
- Git: branch, SHA, last commit
- Session: 279 exchanges summarized
- Topics, done/pending counts
- Last 3 raw messages for detail

Storage

- latest.json (always current)
- lake/YYYY-MM-DD/ (history)
- context.json + manifest.md
- latest.md (readable mirror)

Design Principles

- Zero dependencies (stdlib only)
- File-based -- survives anything
- Generic -- works in any project
- Heuristic summary (no LLM cost)
- Autonomous -- no user action needed

Architecture Block Diagram

Context Lake - Cross-Session Context Persistence Engine

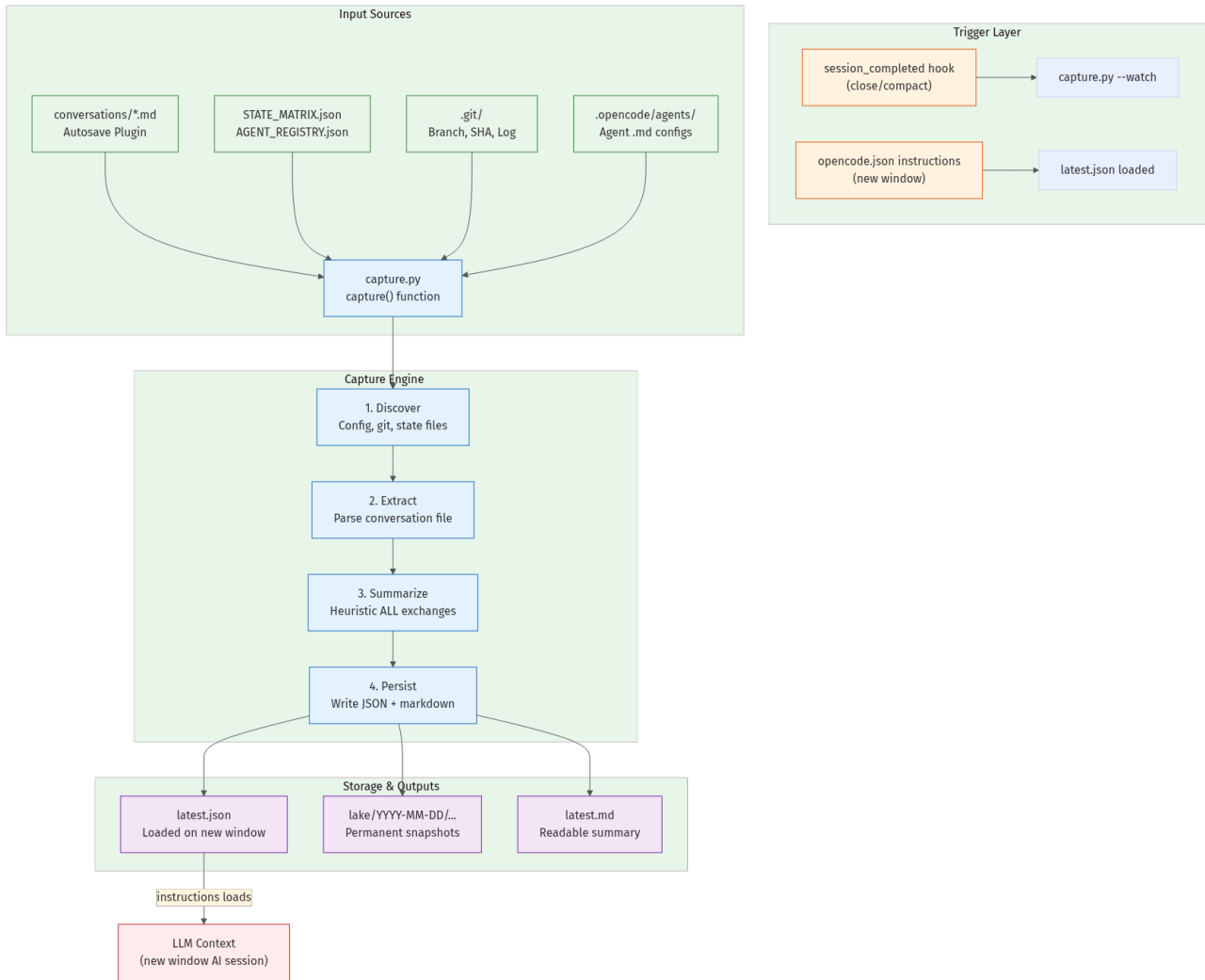


Table of Contents

1. Overview & Purpose
2. Architecture Principles
3. Conversation Plugin (Autosave)
4. Capture Engine (capture.py)
5. Storage Structure
6. Two Capture Modes
7. Heuristic Session Summary
8. Autonomous Triggers
9. Data Flow & Survivability
10. Key Design Decisions
11. File Inventory
12. Configuration Reference
- Appendix A: .drawio File Format

1. Overview & Purpose

Context Lake solves a fundamental problem in session-based AI coding assistants: every new conversation window or compacted session starts with zero context about the project's state, progress, or history. Standard AI assistants have no built-in mechanism to persist context across sessions.

The Lake is a file-based persistence layer that captures the following automatically:

- Pipeline progress: how many agents completed, current phase, last milestone
- Individual agent state: status, artifacts, compliance, review feedback
- Git state: branch, HEAD SHA, last commit message, remote URL
- Full conversation summary: session goal, topics discussed, done/pending counts
- Latest raw messages: last 3 user+assistant exchanges for detail
- Environment: OS, Python version, project root, capture timestamp
- Historical snapshots: permanent lake entries on every session close/compaction

The system operates silently and autonomously. No user interaction is required after initial setup. Context is available in every new window, survives compaction, reboot, and crash.

2. Architecture Principles

2.1 File-Based, Zero Dependencies

All data is stored as JSON and Markdown files on disk. No database, no server, no cloud service. The core engine uses only Python standard library modules: os, sys, json, glob, re, datetime, subprocess, sqlite3, platform, argparse, textwrap. Zero pip installs required. This guarantees portability across any Python 3.10+ environment on any operating system.

2.2 Generic by Design

Config.json uses glob patterns, not hardcoded paths. The engine auto-discovers state files, agent configs, conversation directories, and git repos. The same code works in ANY opencode project with zero configuration changes.

2.3 Silent Operation

The session_completed hook in opencode.json runs capture.py automatically on every compaction or window close -- with no console popup, no user interaction, and zero visual feedback. The capture runs silently in-process.

2.4 Survives Everything

Because everything is on disk, the Lake survives: conversation compaction, window close without save, IDE crash, operating system reboot, power loss. The latest.json file is overwritten atomically (write + close).

3. Conversation Plugin (Autosave)

The conversation plugin 'opencode-autosave-conversation' is declared in opencode.json and is the primary source of conversation content for the Lake.

3.1 Plugin Configuration

```
"plugin": ["opencode-autosave-conversation", "@slkiser/opencode-quota"]
```

3.2 What It Saves

- All user messages and assistant responses as Markdown (.md) files
- Tool calls with status, input JSON, and output
- Assistant reasoning blocks inside <details> tags
- Sub-agent delegation conversations
- Timestamps on every message block
- Session metadata (creation time, topic)

3.3 File Format

```
# Session: New session - YYYY-MM-DDT...
**Created:** YYYY-MM-DD HH:MM:SS
---
## Conversation

### [AI] Assistant
*2026-06-02 12:47:46*

user question text...

*[step-start part]*

[Thinking] **Reasoning:**
<details><summary>Click to expand reasoning</summary>
...
</details>

#### [Tool] Tool: tool-name
**Status:** completed
**Input:** ``json ... ``
**Output:** `` ... ``

*[step-finish part]*
```

3.4 Known Limitation

The plugin labels ALL messages as '#### [AI] Assistant' regardless of role. There are zero '#### [User] User' headers in any conversation file. The Context Lake compensates by inferring roles from the position relative to *[step-start]* markers: text before step-start is user, after is assistant.

3.5 File Naming

Files are saved to the 'conversations/' directory with format: YYYYMMDD-HH-MM-SS-Topic.md. This enables chronological sorting by filename alone.

4. Capture Engine (capture.py)

The core engine is a single 839-line Python file at `context_lake/capture.py`. It performs discovery, extraction, summary, and persistence in one pass.

4.1 Entry Point

```
python context_lake/capture.py
python context_lake/capture.py --window "My Window"
python context_lake/capture.py --project-root /path --json
```

4.2 Capture Pipeline (sequential)

The `capture()` function runs seven stages in order:

- 1. Config loading: load and merge `config.json` with defaults
- 2. Window detection: read terminal title, or use `--window` arg
- 3. Project discovery: read git state, OS, Python version
- 4. Session discovery: query opencode SQLite DB for recent sessions
- 5. State file discovery: glob `STATE_MATRIX.json` and `AGENT_REGISTRY.json`
- 6. Agent config discovery: read `.opencode/agents/*.md` files
- 7. Conversation analysis: read latest conversation, extract summary + last messages

4.3 Message Extraction Algorithm

The `extract_clean_messages()` function parses the raw conversation file:

- Skipping session metadata before the first '###' header
- Tracking `<details>` nesting to skip reasoning blocks
- Using `*[step-start*]` markers to distinguish user vs assistant
- Skipping tool call metadata (lines starting with `####`, `<`, `>`)
- Filtering blocks shorter than 10 characters (noise)
- Capping individual blocks at 5000 characters (avoids subagent dump bloat)

5. Storage Structure

5.1 Directory Layout

```
context_lake/
  config.json      # Generic config (glob patterns, intervals)
  capture.py       # Core engine (839 lines, zero deps)
  latest.json      # Always-fresh snapshot (auto-loaded on new window)
  latest.md        # Human-readable mirror of latest.json
  lake/           # Permanent historical entries
    YYYY-MM-DD/
      HH-MM-SS_WindowName/
        context.json # Full structured JSON
        manifest.md  # Human-readable summary
  hooks/
    capture-on-agent.bat # Agent completion hook
```

5.2 latest.json Schema

```
{
  "latest_entry": "lake/2026-06-02/14-47-03_Test-Summary-v2",
  "window_name": "Test-Summary-v2",
  "captured_at": "2026-06-02T10:16:57+00:00",
  "git_branch": "develop",
  "git_sha": "486bb6e",
  "last_commit": "fix(qa-dashboard): ...",
  "conversation": {
    "file": "20260602-08-15-02-....md",
    "exchanges": 279,
    "goal": "what is now latest status...",
    "last_query": "good then. i will reload...",
    "last_response": "## Commit Report...",
    "topics": ["admin", "api", "auth", "ci", "deploy", ...],
    "done_mentions": 94,
    "pending_mentions": 444
  },
  "state_files": [...],
  "agent_count": 19,
  "recent_sessions": [...]
}
```

5.3 Permanent Entry (context.json)

Same structure plus: full conversations array with all 6 files, complete git log (last 20 commits), opencode sessions (up to 100), full agent configs (19).

6. Two Capture Modes

The Lake has exactly two capture modes. Neither requires user action -- both fire autonomously in normal workflows.

6.1 Session Completed Hook (Primary)

Triggered by `opencode.json` `experimental.hook.session_completed` on every compaction or window close:

```
"experimental": {
  "hook": {
    "session_completed": [
      {
        "command": [
          "python", "context_lake/capture.py",
          "--window", "session-completed",
          "--project-root", "."
        ]
      }
    ]
  }
}
```

Creates a PERMANENT lake entry (`lake/YYYY-MM-DD/HH-MM-SS_.../`) with the full structured snapshot. Also updates `latest.json` and `latest.md`.

6.2 Manual Capture (On Demand)

`python context_lake/capture.py --window "Custom Name"`. Creates permanent entry. Used for manual checkpoints, agent completion callbacks, or testing.

7. Heuristic Session Summary

Rather than truncating conversations to the last N messages (which loses context), the Lake generates a heuristic summary of ALL exchanges in the session. No LLM is needed -- pure regex and counting.

7.1 Summary Fields

- Goal: first user message in the session (captures intent)
- Last query: most recent user message (captures current task)
- Last response: last assistant response truncated to 300 chars
- Topics: extracted via regex from assistant responses (15 topic keywords)
- Done mentions: count of 'done|complete|finished|implemented|built|added|fixed'
- Pending mentions: count of 'next|pending|remaining|still|yet|todo|blocked|bug|issue'
- Exchanges: total user+assistant turns in the session

7.2 Example Output

```
Session Summary:
File: 20260602-08-15-02-....md
Goal: 'what is now latest status from session_Chcekpoint...'
Topics: admin, ai, api, auth, ci, cd, deploy, commit, test
Done: 94 | Pending: 444
Last: 'good then. i will reload shortly'
'## Commit Report | | Result | [OK] Commit successful...'
```

7.3 Role Detection (Without Plugin Support)

Since the plugin labels everything as 'Assistant', the Lake infers roles using the `*[step-start]` marker. Text before `step-start` is user, after is assistant. This correctly labels all 279 exchanges in a typical session.

8. Autonomous Triggers

8.1 On Session Close/Compaction

The `experimental.hook.session_completed` fires automatically on every session close or compaction. This is configured once in `opencode.json` and requires no user action. It triggers the capture engine which writes a permanent lake entry and updates `latest.json`.

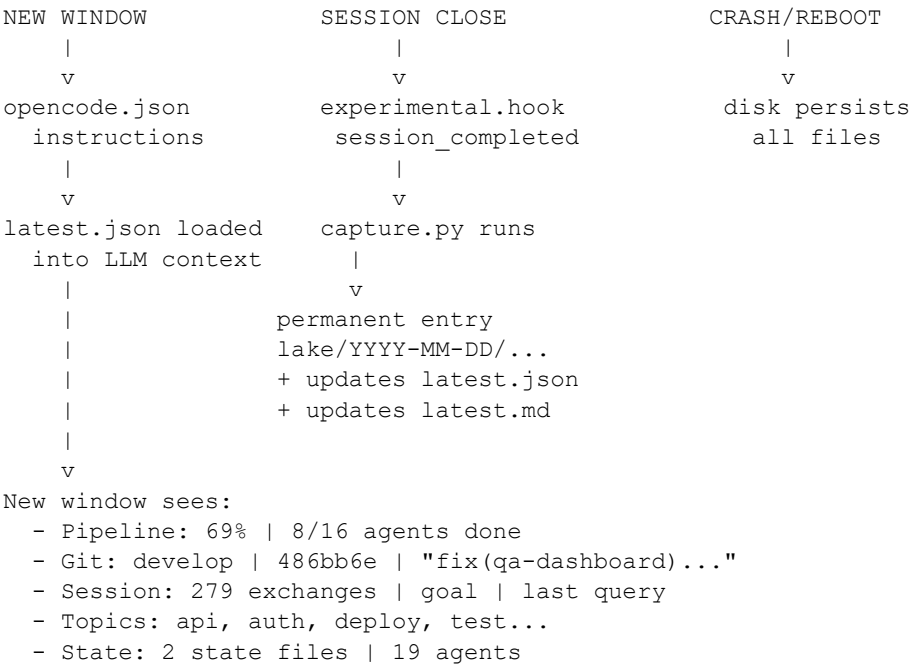
8.2 On New Window Open

```
"instructions": ["context_lake/latest.json"]
```

Every new `opencode` window loads `latest.json` as context. The file is read once at window creation time. The LLM sees: pipeline progress, git state, conversation summary (goal, topics, done/pending counts), agent statuses, and recent sessions.

9. Data Flow & Survivability

The complete lifecycle of context through the Lake:



Survivability Matrix

Event	Context Lost?	Recovery
Window close	No	Permanent lake entry + latest.json updated
Compaction	No	Permanent lake entry + latest.json updated
Crash (unsaved)	No	latest.json from last session_completed
Reboot	No	Disk persists all files
New window	No	latest.json auto-loads via instructions

10. Key Design Decisions

10.1 File-based over DB

No SQLite, no server. Files survive compaction, crash, and reboot. The opencode DB was initially targeted for session tracking but was absent on this machine. The Lake degrades gracefully.

10.2 latest.json over lake directory per capture

Only meaningful events (session close, manual capture) create permanent lake entries. latest.json is always the single source of truth.

10.3 Python stdlib over external packages

Zero pip installs. All stdlib modules (os, sys, json, glob, re, datetime, etc). Works on any Python 3.10+ environment immediately.

10.4 Heuristic summary over LLM summary

Regex + counting is instant, free, deterministic. No tokens consumed, no latency. Captures session goal, topics, done/pending indicators.

10.5 Filename sort over mtime sort

Conversation files sorted by filename descending = chronologically. The naming convention (YYYYMMDD-HH-MM-SS) makes this reliable.

10.6 Session_completed hook over watch/background process

OpenCode does not support on_start hooks. session_completed fires on every close/compact. Combined with instructions for on-open loading, this covers both critical moments. No background process needed.

11. File Inventory

context_lake/capture.py (839 lines)

Core capture engine.

context_lake/config.json (33 lines)

Generic configuration.

context_lake/latest.json (~30 lines)

Always-fresh snapshot.

context_lake/latest.md (~40 lines)

Human-readable mirror.

context_lake/lake/.../context.json (varies)

Permanent history.

context_lake/lake/.../manifest.md (varies)

Permanent summary.

context_lake/hooks/capture-on-agent.bat (5 lines)

Agent completion hook.

context_lake/LAKE_ARCHITECTURE.drawio (editable)

Architecture diagram.

opencode.json (29 lines)

Plugin, instructions, hook config.

00_state_ledger/STATE_MATRIX.json (1564 lines)

Pipeline state.

00_state_ledger/AGENT_REGISTRY.json (~50 lines)

Agent definitions.

conversations/*.md (various)

Full conversation files.

12. Configuration Reference

config.json

```
{
  "lake": {
    "max_entries_per_dir": 1000,
    "date_format": "%Y-%m-%d",
    "datetime_format": "%Y-%m-%dT%H-%M-%S"
  },
  "capture": {
    "state_file_patterns": [
      "**/STATE_MATRIX*.json",
      "**/*_STATE*.json",
      "**/*_REGISTRY*.json",
      "**/state_ledger/**/*.json",
      "**/state/**/*.json"
    ],
    "conversation_dirs": ["conversations"],
    "agent_dirs": [".opencode/agents"],
    "git": {"enabled": true, "max_log_entries": 20}
  },
  "entry_name_template": "{datetime}_{window_name}"
}
```

opencode.json (relevant sections)

```
{
  "plugin": ["opencode-autosave-conversation"],
  "instructions": ["context_lake/latest.json"],
  "experimental": {
    "hook": {
      "session_completed": [
        {
          "command": ["python", "context_lake/capture.py",
            "--window", "session-completed",
            "--project-root", "."]
        }
      ]
    }
  }
}
```

Key Configuration Points

- `state_file_patterns`: glob patterns for discovering pipeline state files
- `conversation_dirs`: where to look for autosave conversation files
- `agent_dirs`: directory containing opencode agent .md definitions
- `git.max_log_entries`: number of recent commits to capture (20 default)
- `entry_name_template`: format for permanent lake folder names
- All paths are relative to project root

Appendix A: .drawio Architecture File

An editable draw.io version of the architecture block diagram is included at `context_lake/LAKE_ARCHITECTURE.drawio`. Open it in draw.io (app.diagrams.net) or VS Code with the Draw.io extension to view, edit, and export the diagram.

The diagram shows the three-layer architecture: Input Sources (conversation files, state files, git, agent configs) feeding into the Capture Engine with its four internal stages (Discover, Extract, Summarize, Persist), which writes to Storage (`latest.json`, `lake/`, `latest.md`). The trigger layer (`session_completed` hook, instructions) orchestrates the flow.